SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	G. Akshya	**Reg. No.:**	192425132
Date:	27/8/2026	**Duration:**	60 minutes

Code of Conduct

I G. Akshya 192425132 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Akshya

Co 4 Assessment Tool 02

Name: -G. Akshya **Reg No:** -192425132

Course Code: -CSA1007 **Slot:** -D

1. Problem Statement and Project Overview

Smart Vertical Farming Automation Platform is a software and IoT-based system designed to manage a controlled vertical farming environment. The platform monitors environmental conditions, optimizes irrigation, predicts crop health, automates lighting control, generates production reports, and notifies farm operators.

For this problem statement, a CI/CD pipeline is required so that changes to the monitoring dashboard, backend services, prediction modules, APIs and automation logic can be integrated, tested, checked for quality and securely deployed with minimum manual effort.

2. CI/CD Requirements Analysis

CI/CD Need	Requirement for the Farming Platform
Frequent code integration	Developers may modify dashboard, APIs, ML logic or automation rules; changes should be integrated through version control.
Automated build	Dependencies and application packages should be built consistently after each valid code change.

Automated testing	Unit, API/integration and selected system tests should run automatically.
Quality and security	Code style, static analysis and dependency/security checks should block unsafe builds.
Containerized packaging	A versioned Docker image can provide a repeatable deployment unit.

3. Proposed CI/CD Pipeline Architecture

The proposed pipeline follows the sequence Source Control → CI Trigger → Build → Automated Testing → Quality and Security Checks → Package → Staging Deployment → Approval → Production Deployment → Monitoring and Rollback. A successful stage is required before the pipeline proceeds to the next stage.

4. Application Architecture Relevant to CI/CD

The software system can be divided into sensor/IoT integration, backend services, decision and prediction logic, dashboard services, database services and actuator-control interfaces. The CI/CD pipeline validates the software portions of these modules before deployment.

5. Pipeline Stages and Workflow

Stage	Suggested Tool/Technology	Purpose
1. Source Code Management	GitHub/Git	Stores application source code, configuration and pipeline files. Feature branches and pull requests support controlled integration.
2. CI Trigger	GitHub Actions	Starts the pipeline on push or pull request events.
3. Build	Python/Node environment + Docker	Installs dependencies, validates configuration and builds the application/container.

4. Unit Testing	pytest / frontend test framework	Tests individual functions such as sensor processing, threshold logic, report generation and prediction utilities.
5. Integration/API Testing	pytest + API test tools	Checks communication between backend services, database interfaces and APIs.
6. Code Quality	Ruff/Flake8 + formatting checks	Detects style problems, unused code and common programming issues.

6. Automated Testing Strategy

- Unit testing: validate sensor-data processing, threshold calculations, irrigation decisions, lighting rules, report calculations and prediction helper functions.
- Integration testing: verify API endpoints, database operations and communication between backend modules.
- Hardware-interface mocking: use mocked sensor and actuator interfaces during CI so that software tests do not require physical farm hardware.
- Regression testing: run previously successful tests on every important change to prevent existing features from breaking.

7. Quality and Security Integration

- Static code analysis should run automatically and fail the pipeline when configured critical issues are detected.
- Dependency scanning should identify vulnerable third-party packages before deployment.
- Secrets such as database passwords, API keys and cloud credentials should be stored in protected CI/CD secrets rather than source code.
- Pull requests should require successful CI checks before merging into the main branch.
- Docker images should be tagged with a unique version or commit identifier to support traceability.
- Production deployment permissions should be restricted to authorized users or protected environments.

8. Deployment Strategy

Environment	Purpose	Deployment Rule
Development	Used by developers for feature development and local testing.	Feature branches / development deployment.
CI Validation	Temporary pipeline environment for build, unit tests, quality and security checks.	Only validated code proceeds.
Staging	Production-like environment for integration and smoke testing.	Candidate release is verified before production.
Production	Live smart farming monitoring and automation platform.	Protected deployment with monitoring and rollback.

9. Deployment and Rollback Process

1. Developer creates a feature branch and commits the change.
2. Pull request triggers build, unit tests, integration tests, quality checks and security scans.
3. If any required check fails, the pipeline stops and the developer receives a failure notification.
4. After successful validation, a versioned Docker image is created.
5. The image is deployed to staging and smoke tests are executed.

10. Notifications and Monitoring

- ❖ Notify developers when builds or tests fail.
- ❖ Notify the responsible team when staging or production deployment succeeds or fails.
- ❖ Record build number, commit identifier, image version and deployment status for traceability.
- ❖ Monitor API availability, application logs and important service-health indicators.
- ❖ Keep deployment logs so that failures can be analyzed and corrected.

11. Advantages of the Proposed CI/CD Pipeline

- ➢ Faster and more reliable integration of software changes.
- ➢ Reduced manual testing and deployment effort.
- ➢ Early detection of software defects and security issues.
- ➢ Consistent packaging through containerization.

- Safer separation between development, staging and production.
- Easy traceability through versioned source code and images.

12. Limitations and Considerations

- CI tests cannot completely replace physical hardware testing.
- Sensor and actuator behavior should also be validated in a controlled hardware test environment.
- Cloud or network failures may affect deployment and monitoring.
- ML model changes require additional validation beyond ordinary software unit tests.
- Production automation should use strong access control because an incorrect deployment could affect real irrigation or lighting equipment.

13. Tools and Technology Justification

Tool / Technology	Justification
Git/GitHub	Version control, pull requests and collaboration.
GitHub Actions	Integrated CI/CD automation suitable for repository-based workflows.
Python + pytest	Suitable for backend logic, automation rules and automated tests.
Docker	Provides repeatable, versioned application packaging.
Static analysis tools	Improve maintainability and identify common defects early.
Dependency/SAST scanning	Adds security checks before deployment.
Staging environment	Allows final application validation before production.
Protected production environment	Reduces accidental production deployments.

14. Conclusion

The proposed CI/CD pipeline provides a structured DevOps workflow for the Smart Vertical Farming Automation Platform. It automates source integration, building, testing, quality analysis, security checking, packaging and deployment. The use of development, staging and production environments provides controlled release management, while versioned packages and rollback mechanisms improve reliability.

The pipeline is particularly important for this problem because the platform combines software services with real-world farming automation. Changes to irrigation, lighting, crop-health prediction or monitoring must be tested carefully before they are allowed to reach production. Therefore, automated quality checks, security controls, staged deployment and rollback provide a safer approach to continuously evolving the system.

15. Problem Statement Objectives

1. Monitor environmental conditions.
2. Optimize irrigation.
3. Predict crop health.
4. Automate lighting control.
5. Generate production reports.
6. Notify farm operators.